

- Note:** 1. All Programs must be in handwritten copies with proper TOC (SNo., Name of Program, Date, Pg No), Page no. at each page etc.
2. All programs output will be from your **Desktop/Laptop/Tab/Mobile** device with showing **your name directory** and execution **date** in each output.
3. Writing format for each program is **question** followed by **code** and **Output Screen on the blank page**.

Sno. Programs

Java Basics & Control Statements

1. Write Java Program to print **Hello World**
2. Write Java Program to display **student details**
3. Write Java Program to **add two numbers**
4. Write Java Program to **swap two numbers**
5. Write Java Program to swap two numbers **without using third variable**
6. Write Java Program to **check whether a number is even or odd**
7. Write Java Program to **check whether a number is positive, negative or zero**
8. Write Java Program to **find the largest of two numbers**
9. Write Java Program to **find the largest of three numbers**
10. Write Java Program to **calculate Simple Interest**
11. Write Java Program to **calculate Compound Interest**
12. Write Java Program to **calculate area of a circle**
13. Write Java Program to **calculate perimeter of a rectangle**
14. Write Java Program to **convert Celsius to Fahrenheit**
15. Write Java Program to **check whether a year is a leap year**
16. Write Java Program to **check whether a number is prime**
17. Write Java Program to **check whether a number is palindrome**
18. Write Java Program to **check whether a number is Armstrong**
19. State the order of evaluation of the operations in each of the following Java statements and implement them to show the value of x after each statement. $x = 7 + 3 * 6 / 2 - 1$; $x = 2 \% 2 + 2 * 2 - 2 / 2$; $x = (3 * 9 * (3 + (9 * 3 / (3))))$;
20. Write an application that calculates the squares and cubes of the numbers from 0 to 10 and prints the resulting values in a table format, as shown below.

Number	Square	Cube
0	0	0
1	1	1
2	4	8 and so on
21. A mail-order house sells five products whose retail prices are as follows : Product 1 : Rs. 99.90 , Product 2 : Rs. 20.20 , Product 3 : Rs. 6.87 Product 4 : Rs. 45.50 and Product 5 : Rs. 40.49 . Write an application that reads a series of pairs of numbers as follows :
a) product number b) quantity sold
your program use a *switch* statement to determine the retail price for each product. It should calculate and display the total retail value of all products sold.
22. write application that take a series of integers in pair(e.g. (a,b)) and find the following
i) Is first one is greater ii) Is both are even iii) They are relatively prime or not

Array & String Class

23. Write Java Program to **find largest and smallest element in an array**
24. Write Java Program to **find sum and average of array elements**
25. Write Java Program to **reverse an array**
26. Write Java Program to **count frequency of elements in an array**
27. Write Java Program to **sort array using Bubble Sort**
28. Write Java Program to **sort array using Selection Sort**
29. Write Java Program to **perform Binary Search**
30. Write Java Program to **add two matrices**
31. Write Java Program to **transpose a matrix**
32. Write Java Program to **reverse a string**
33. Write Java Program to **check palindrome string**
34. Write Java Program to **count vowels and consonants in a string**
35. Write an application that uses *String* method *compareTo* to compare two strings defined by the user.
36. Write an application that uses *String* method *equals* and *equalsIgnoreCase* to tests any two string objects for equality.
37. Write an application that uses *String* method *indexOf* to determine the total number of occurrences of any given alphabet in a defined text.
38. Write an application that uses *String* method *concat* to concatenate two defined strings.
39. Write an application that finds the length of a given string.
40. Write an application that uses *String* method *charAt* to reverse the string.
41. Write an application that finds the substring from any given string using *substring* method and *startsWith* & *endsWith* methods.
42. Write an application that changes any given string with uppercase letters, displays it , changes it back to lowercase letters and displays it.

Class, Method and Object

43. Create a class called *Employee* that includes three pieces of information as instance variables – a first name (type *String*), a last name (type *String*) and a monthly salary (double)
44. Create a constructor in above class to initialize the three instance variables. Provide a *get* method for each instance variable.
45. Write a test application named *EmployeeTest* that demonstrates class *Employee*'s capabilities. Create two employee objects and display each object's yearly salary.
46. Create a class *Account* with an instance variable *balance* (double). It should contain a constructor that initializes the *balance*, ensure that the initial balance is greater than 0.0.

- Note:** 1. All Programs must be in handwritten copies with proper TOC (SNo., Name of Program, Date, Pg No), Page no. at each page etc.
2. All programs output will be from your **Desktop/Laptop/Tab/Mobile** device with showing **your name directory** and execution **date** in each output.
3. Writing format for each program is **question** followed by **code** and **Output Screen on the blank page**.

47. Create two methods namely *credit* and *getBalance*. The first one adds the amount (passed as parameter) to balance and does not return any data. The second method allows clients (i.e. the other classes that use this class) to obtain the value of a particular *Account* object's *balance*.
48. Create class *AccountTest* to create and manipulate an *Account* object.
49. Write another method *debit* in the above program that withdraws money from an *Account*. Ensure that the debit amount does not exceed the *Account*'s *balance*. In that case the *balance* should be left unchanged and the method should print a message indicating "Debit amount exceeded account balance". Modify class *AccountTest* to test method *debit*.

Pattern

50. Write Java Program to **print right triangle star pattern**
51. Write Java Program to **print inverted right triangle star pattern**
52. Write Java Program to **print pyramid star pattern**
53. Write Java Program to **print inverted pyramid star pattern**
54. Write Java Program to **print diamond star pattern**
55. Write Java Program to **print hollow square star pattern**
56. Write Java Program to **print butterfly star pattern**
57. Write Java Program to **print sandglass star pattern**
58. Write Java Program to **print number triangle pattern**
59. Write Java Program to **print Floyd's triangle**
60. Write Java Program to **print Pascal's triangle**
61. Write Java Program to **print alphabet triangle pattern**
62. Write Java Program to **print spiral number pattern**
63. Write Java Program to **print concentric square pattern**
64. Write Java Program to **print right triangle star pattern**
65. Write Java Program to **print inverted right triangle star pattern**
66. Write Java Program to **print pyramid star pattern**
67. Write Java Program to **print inverted pyramid star pattern**

Inheritance

68. Write an application to create a super class *Employee* with information first name & last name and methods *getFirstName()*, *getLastName()* derive the sub-classes *ContractEmployee* and *RegularEmployee* with the information about department, designation & method *displayFullName()*, *getDepartment()*, *getDesignation()* to print the salary and to set department name & designation of the corresponding sub-class objects respectively.
69. **Derive sub-classes of *ContractEmployee* namely *HourlyEmployee* & *WeeklyEmployee* with information number of hours & wages per hour, number of weeks & wages per week respectively & method *calculateWages()* to calculate their monthly salary. Also override *getDesignation()* method depending on the type of contract employee.**
70. Write an application to create a super class *Vehicle* with information vehicle number, insurance number, color and methods *getConsumption()* and *displayConsumption()*. Derive the sub-classes *TwoWheeler* and *FourWheeler* with method *maintenance()* and *average()* to print the maintenance and average of vehicle.
71. Extend the above *TwoWheeler* class with methods *getType()* and *getName()* which gives the information about the type and the name of the company. Create sub-classes *Geared* and *NonGeared* with method *average()* to print the average of a geared and non-geared two wheeler.
72. Create a super class *CommunityMember* with the information of member i.e. name, address, contact, date_of_join, through methods *getName()*, *getAddress()*, *getContact()*, *getDate_of_Join()* and derive sub-classes *Employee* and *Student* with method *Qualification()* to print the related information with his/her qualification.
73. Create a super class *Shape* with methods *getName()* which gives the information about the type of the shape. derive its sub-classes *TwoDim* and *ThreeDim* with method *area()* and *volume()* respectively which prints the area and volume of a two-dimensional and three-dimensional shape.
74. Extend the class *TwoDim* with methods *getLength()*, *getBreadth()* which displays the length and breadth of two dimensional shapes. Derive sub-classes *rectangle*, *rhombus* with method *getArea()* and *getPerimeter()* to calculate the area and perimeter of this two dimensional shapes.
75. Extend the class *ThreeDim* with methods *getLength()*, *getBreadth()*, *getHeight()* which displays the length, breadth and height of three dimensional shapes. Derive sub-classes *cuboid*, *tetrahedron* with method *getArea()* and *getVolume()* to calculate the area and volume of this three dimensional shapes.
76. Create a super class *Student* with methods *getQual()*, *getFirstName()*, *getLastName()*, *getAddress()*, *getContact()*, which gives basic details of student. derive sub-classes *Faculty* and *Scholar* with method *salary()*, *Course()* resp. which gives the additional information about the salary and course of faculty and scholar resp. .
77. Create an abstract class *Shape* which calculate the area and volume of 2-d and 3-d shapes with methods *getArea* and *getVolume*. Reuse this class to calculate the area and volume of square, circle, cube.
78. Create an abstract class *Employee* with methods *getAmount()* which displays the amount paid to employee. Reuse this class to calculate the amount to be paid to *WeeklyEmployee* and *HourlyEmployee* according to no. of hours and total hours for *HourlyEmployee* and no. of weeks and total weeks for *WeeklyEmployee*.
79. Create an Interface *payable* with method *getAmount()*. Calculate the amount to be paid to Invoice and Employee by implementing Interface.
80. Create an Interface *Vehicle* with method *getColor()*, *getNumber()*, *getConsumption()*. Calculate the fuel consumed, name and color for *TwoWheeler* and *FourWheeler* by implementing interface *Vehicle*.
81. Create an Interface *Fare* with method *getAmount()* to get the amount paid for fare of travelling. Calculate the fare paid by bus and train implementing interface *Fare*.
82. Create an Interface *StudentFee* with method *getAmount()*, *getFirstName()*, *getLastName()*, *getAddress()*, *getContact()*. Calculate the amount paid by the Hostler and NonHostler student by implementing interface *StudentFee*
83. Write Java Program to create your own package. Package should have more than two classes. Write a class that uses the package.
84. Create a package named *org.shapes*. Create some classes in the package representing some common geometric shapes like Square, Triangle, Circle and so on.

- Note:** 1. All Programs must be in handwritten copies with proper TOC (SNo., Name of Program, Date, Pg No), Page no. at each page etc.
2. All programs output will be from your **Desktop/Laptop/Tab/Mobile** device with showing **your name directory** and execution **date** in each output.
3. Writing format for each program is **question** followed by **code** and **Output Screen on the blank page**.

Exception Handling

- 85. Exception Handling program for division of two numbers that accepts numbers from user.
- 86. Exception Handling program for storing values in array of *int* or *String* that results into buffer overflow
- 87. Exception Handling program for *NullPointerException*--thrown if the JVM attempts to perform an operation on an *Object* that points to no data, or *null*
- 88. Exception Handling program for *NumberFormatException*--thrown if a program is attempting to convert a string to a numerical datatype, and the string contains inappropriate characters (i.e. 'z' or 'Q')
- 89. Exception Handling program for *ClassNotFoundException*--thrown if a program can not find a class it depends at runtime (i.e., the class's ".class" file cannot be found or was removed from the CLASSPATH)
- 90. Exception Handling program for *IOException*--actually contained in *java.io*, but it is thrown if the JVM failed to open an I/O stream
- 91. Write a program that shows that the order of the catch blocks is important. If you try to catch a superclass exception type before a subclass type, the compiler should generate errors.
- 92. Program for demonstrating the use of throw, throws & finally - Create a class with a main() that throws an object of class Exception inside a try block. Give the constructor for Exception a String argument. Catch the exception inside a catch clause and print the String argument. Add a finally clause and print a message to prove you were there.
- 93. Create your own exception class using the extends keyword. Write a constructor for this class that takes a String argument and stores it inside the object with a String reference. Write a method that prints out the stored String. Create a try-catch clause to exercise your new exception.
- 94. Write a program to rethrow an exception – Define methods one() & two(). Method two() should initially throw an exception. Method one() should call two(), catch the exception and rethrow it Call one() from main() and catch the rethrown exception.

Multithreading

- 95. Write Java Program to **create thread using Thread class**
- 96. Write Java Program to create thread that print counting by extending Thread class.
- 97. Write Java Program to **create thread using Runnable interface**
- 98. Write Java Program to **demonstrate thread priority**
- 99. Write Java Program to **demonstrate synchronization**
- 100. Write Java Program to **implement producer-consumer problem**
- 101. Write Java Program to determine whether a thread is alive or not.
- 102. Write Java Program to demonstrate join() method of Thread class.
- 103. Write Java Program using thread to create Remainder

File Handling/Package

- 104. Write Java Program to create a file.
- 105. Write Java Program to delete a file.
- 106. Write Java Program to determine a file or dir exist or not.
- 107. Write Java Program to determine whether a file or dir.
- 108. Write Java Program to exit from a Frame window when we click on Close button.
- 109. Write Java Program to move a file.
- 110. Write Java Program to read a file using FileReader and break the contents using StringTokenizer.
- 111. Write Java Program to write into a file using FileWriter.
- 112. Write Java Program to demonstrate RandomAccessFile.
- 113. Write Java Program to rename a file.
- 114. Write Java Program to get Host name of given IP address.
- 115. Write Java Program to get file length.
- 116. Write Java Program to reverse a string by word.
- 117. Write Java Program to demonstrate StringTokenizer class.
- 118. Write Java Program create your own package in defined a class StringUtils in this package.

Networking/ Serialize & Deserialize objects

- 119. Write Java Program to implement toString() method in your class to print objects.
- 120. Write Java Program to send a UDP packet on specified host and port.
- 121. Write Java Program to receive a UDP packet.
- 122. Write Java Program to using pow() function.
- 123. Write Java Program to display system properties
- 124. Write Java Program to get machine user name.
- 125. Write Java Program to demonstrate java.lang.reflect package.
- 126. Write Java Program to determine whether an entered number is prime or not.
- 127. Write Java Program to serialize an object.
- 128. Write Java Program to deserialize an object.

GUI / AWT / Applet

- 129. Create an application to draw a horizontal line.
- 130. Create an application to draw one line perpendicular to other. One line parallel to other.
- 131. Create an application to display a circle within rectangle
- 132. In the above application fill different colors in the circle & rectangle.
- 133. Write an application that displays any string. Choose color from combo box to change the color of this displayed string.
- 134. Write Java Program to demonstrate AWT buttons with event handling.
- 135. Write Java Program to demonstrate BorderLayout in AWT window.
- 136. Write Java Program to change background color according to selected color from combo box.
- 137. Write a program in java to scroll a string using Applet.

- Note:** 1. All Programs must be in handwritten copies with proper TOC (SNo., Name of Program, Date, Pg No), Page no. at each page etc.
2. All programs output will be from your **Desktop/Laptop/Tab/Mobile** device with showing **your name directory** and execution **date** in each output.
3. Writing format for each program is **question** followed by **code** and **Output Screen on the blank page**.

- 138. Write Java Program to demonstrate all mouse events.
- 139. Write Java Program to demonstrate all keyboard events.
- 140. Write Java Program to demonstrate GridLayout.
- 141. Write Java Program to create an Applet having status bar.

Miscellaneous

- 142. Write Java Program to create a chat client.
- 143. Write Java Program to create a chat server.
- 144. Write Java Program to demonstrate Color class.
- 145. Write Java Program to event handling using adapter classes.
- 146. Write Java Program to demonstrate Check Boxes in applet.
- 147. Write Java Program to demonstrate Calander class.
- 148. Write a Java program that randomly fill a 3 by 4 by 6 array and then prints the largest and smallest values in the array.
- 149. Write Java Program to demonstrate String class operations.
- 150. Write Java Program to display all available fonts in an applet window
- 151. Write Java Program to store student objects in a file.
- 152. Write Java Program to display specific student record based on rollnumber. If not found than display all records in tabular form.
- 153. Write Java Program to copy the content of a file to another file.
- 154. Write Java Program to demonstrate Random class.
- 155. Write Java Program to create an Applet that's background color will be change on each second.
- 156. Write Java Program to sum two 2-D matrix and store the result into third matrix.
- 157. Create a program that will print every argument given on the command line. consider how your program will deal with no argument.
- 158. Write Java Program to draw a string and choose its size respectively from combo box.
- 159. Write Java Program to demonstrate data entry program.
- 160. Create an application of cash withdrawal from the bank account that have no. of users that are operating the accounts.(synchronization)
- 161. Write Java Program to create three text boxes and save entered value into a file.
- 162. Implement a class Reader that count the number of times a particular character, such as e, is read. The character can be specified when the stream is created.
- 163. Construct a program Wc ("word count"), which counts number of chars, words and lines of the text file. Space is counted as a character. Empty rows are counted as lines. "Word" will represent a string.
- 164. Write a small application with a default date 01/01/2000 and three combo boxes displaying valid days, months & year(1990 – 2050). Change the displayed date with the one chosen by user from these combo boxes.
- 165. Create a GUI with a text field and three buttons. When you press each button, make some different text appear in the text field.
- 166. Create a GUI application to take input of two numbers(text field) from user. When you press button it should display sum of the two numbers in a third text box.
- 167. Create an applet with a Button and a TextField. Write a referenceEvent() so that if the button has the focus, characters typed into it will appear in the TextField.
- 168. Write an application to create a GUI with two buttons such that clicking on the first displays the message "Welcome to SCS" on the window and clicking on the second changes the color of the message(*hint : toggle the color*)
- 169. Create a GUI with title STUDENT which has labels roll no., name, class, address with textboxes for taking input from the user(without any functionality).
- 170. Create a GUI application for fees receipt which contains checkboxes for selecting the course, radio buttons for selecting gender and labels and corresponding textboxes for name, class, date and amount paid.
- 171. Create a GUI application to display a calculator using grid Layout (You do not have to provide functionality).
- 172. WAP for string tokenizer.
- 173. WAP that generate a random number (1 – 10000). Let the user guess the correct number. User will enter the digit. Program should let the user that input is right or wrong. No of turns that user can make a choice for input is twice the number of digits in the system generated numbers.
- 174. Convert the input date in words. Input format is dd mm yy.